
NVIDIA-CERTIFIED PROFESSIONAL

OpenUSD Development Practice Test

70 questions • 120 minutes • weighted by official exam domains

Difficulty	Mixed
Seed	4217 (reproducible)
Passing target	70% or higher

This practice test mirrors the question style, multi-select format, and domain weighting of the official NVIDIA-Certified Professional: OpenUSD Development exam. Questions and answer key are generated from a curated bank grounded in the Learn OpenUSD curriculum. Not affiliated with or endorsed by NVIDIA; for study use only.

Instructions

This test contains **70 questions** distributed across the eight exam domains by their official weighting. Each question is single-choice (choose one) unless it is explicitly marked (*Select two options.*) or (*Select three options.*), in which case you must select exactly that many answers for credit. Code questions present one or more USDA layers; reason through composition using **LIVERPS** strength order (Local, Inherits, Variants, rElocates, References, Payloads, Specializes). The **Answer Key & Explanations** section begins after the final question.

Domain distribution: Composition 16 · Data Exchange 10 · Pipeline Development 10 · Data Modeling 9 · Debugging and Troubleshooting 8 · Content Aggregation 7 · Visualization 6 · Customizing USD 4

Real exam: 60-70 questions, 120 min, Professional level. Weighting follows the official NVIDIA blueprint.

Question 1

Domain: Visualization

Hydra is described as having three main parts. Which set correctly names all three and their roles?

- A. Scene proxy (provides scene information), render context (tracks changes), render plugin (creates the final image)
- B. Scene delegate (provides scene information), render index (tracks changes and manages the scene), render delegate (creates the final image)
- C. Render index (provides scene information), scene delegate (tracks changes), render delegate (issues draw calls to the GPU)
- D. Scene delegate (creates geometry), shader graph (manages material connections), render backend (generates pixels)

Source: Learn OpenUSD: Hydra — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/hydra.html>

Question 2

Domain: Customizing USD

A developer deactivates the prim at /World/Prop by calling `prim.SetActive(False)`. They then need to access that prim object in Python to inspect or modify its metadata. Which statement correctly describes what is possible?

- A. The developer must call `prim.SetActive(True)` to reactivate the prim before the USD API can access it
- B. `stage.Traverse()` visits /World/Prop because `SetActive(False)` affects rendering decisions, not scenegraph traversal
- C. `stage.GetPrimAtPath("/World/Prop")` returns a valid prim object even though the prim does not appear in a default `stage.Traverse()`
- D. `stage.GetPrimAtPath("/World/Prop")` returns `None` or an invalid prim because inactive prims are removed from the scenegraph

Source: Learn OpenUSD: Active and Inactive Prims — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/active-inactive-prims.html>

Question 3

Domain: Pipeline Development *(Select two options.)*

A studio is organizing assets into parallel workstreams using OpenUSD layer stacks. Which practices are recommended?

- A. Layer stacks should be kept simple, avoiding workstreams that grow procedurally, since each layer has a resolve-and-open cost
- B. Layer stacks should serve as the asset versioning mechanism, with each version captured as a sublayer
- C. Each workstream should author only its unique contributions sparsely rather than overwriting other workstreams' work
- D. Simple assets should be split into separate workstream layers up front so parallelism is available later
- E. Computational workstreams should remain static and consistent from one evaluation of the stage to the next

Source: Learn OpenUSD: Modeling Workstreams With Layer Stacks — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/workstreams/modeling-workstreams.html>

Question 4

Domain: Composition

In the layer below, the reference omits a file identifier and targets `</Master>`. What kind of reference is this and what does it accomplish?

scene.usda

```
#usda 1.0
(
  defaultPrim = "World"
)

def "World"
{
  def "Master"
  {
    def Sphere "Ball"
    {
      double radius = 2
    }
  }

  def "InstanceA" (
    prepend references = </Master>
  )
  {
  }
}
```

- A. An inherit arc that broadcasts /Master's opinions to /World/InstanceA, since no file identifier is present
- B. A payload that defers loading /Master's content until /World/InstanceA is explicitly requested
- C. A subroot reference that fails to resolve, because the layer's defaultPrim is /World rather than /Master
- D. An internal reference that targets a prim in the same layer, reusing /Master's content at /World/InstanceA

Source: Learn OpenUSD: Referencing Basics — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/references.html>

Question 5

Domain: Pipeline Development

Reviewing this layer for model hierarchy validity, the Cube "Body" is a leaf gprim under a component. Which prim, if tagged `kind = component`, would make this model hierarchy invalid?

model_kinds.usda

```
#usda 1.0
(
    defaultPrim = "World"
)
def Xform "World" (
    kind = "group"
)
{
    def Xform "Component" (
        kind = "component"
    )
    {
        def Cube "Body"
        {
        }
    }
}
}
```

- A. Leaving "Body" untagged, because every descendant of a component model must carry the subcomponent kind
- B. Tagging "World" with kind = group, because the ancestors of a component model may not carry the group kind
- C. Tagging "Body" with kind = component, because a component is a leaf model that cannot appear beneath another component model
- D. Tagging "Component" with kind = component, because component models require an assembly parent rather than a group

Source: Learn OpenUSD: What Are Model Kinds? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/what-are-model-kinds.html>

Question 6

Domain: Data Modeling

A translate xformOp has time samples at frames 1 and 24 but no default value. What value does the op resolve to when queried at time code 12?

anim.usda

```
#usda 1.0

def Xform "Mover"
{
    double3 xformOp:translate.timeSamples = {
        1: (0, 0, 0),
        24: (23, 0, 0),
    }
    uniform token[] xformOpOrder = ["xformOp:translate"]
}
```

- A. (11, 0, 0), linearly interpolated between the two samples
- B. (23, 0, 0), the last authored sample
- C. (0, 0, 0), the first authored sample
- D. The schema fallback, because no default value was authored

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 7

Domain: Data Modeling

A pipeline tool procedurally displaces the vertex positions of a mesh each frame without changing its topology. To keep the mesh well-formed for renderers, which pair of attributes must be kept synchronized?

blob.usda

```
#usda 1.0

def Mesh "Blob"
{
    int[] faceVertexCounts = [4, 4]
    int[] faceVertexIndices = [0, 1, 2, 3, 3, 2, 4, 5]
    point3f[] points = [(0,0,0), (1,0,0), (1,1,0), (0,1,0), (2,0,0), (2,1,0)]
    float3[] extent = [(0,0,0), (2,1,0)]
}
```

- A. faceVertexCounts and extent
- B. points and extent
- C. faceVertexCounts and faceVertexIndices
- D. points and faceVertexIndices

Source: Learn OpenUSD: Attributes — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/properties/attributes.html>

Question 8

Domain: Visualization

A `UsdLuxDistantLight` is authored with only `xformOp:rotateXYZ = (45,0,0)` and no other transform. Before any transform is applied, along which local axis does the light emit?

example.usda

```
def DistantLight "Sun"
{
    float3 xformOp:rotateXYZ = (45, 0, 0)
    uniform token[] xformOpOrder = ["xformOp:rotateXYZ"]
}
```

- A. Along the local +Y axis
- B. Along the local +Z axis
- C. Radially in all directions
- D. Along the local -Z axis

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

Question 9

Domain: Pipeline Development

A `PointInstancer` must hide a subset of instances now but animate them back on at later time codes, while a different, permanent set must be pruned and supports sparse list-editing of which ids it affects. Which properties drive each case?

- A. `invisibleIds` for the permanent prune, which can be list-edited; `inactivelds` for the time-animated subset
- B. `inactivelds` for both subsets, with time samples distinguishing the temporary set from the permanent one
- C. `invisibleIds` for the time-animated on/off subset; `inactivelds` for the permanent prune, which can be list-edited

D. protoIndices set to -1 for the hidden subset; the ids array cleared for the permanently pruned set

Source: Learn OpenUSD: Refining Point Instances — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-point-instances.html>

Question 10

Domain: Data Exchange

OBJ stores one global vertex list grouped per face, but UsdGeom.Mesh prefers a flattened index list recomposed via face vertex counts, as in the Mesh below. In a loop over each face's index group, how are faceVertexCounts and faceVertexIndices assembled?

mesh.usda

```
#usda 1.0

def Mesh "Cube"
{
    int[] faceVertexCounts = [4, 4, 4]
    int[] faceVertexIndices = [0, 1, 2, 3, 4, 5, 6, 7, 0, 3, 5, 4]
    uniform token subdivisionScheme = "none"
}
```

- A. Append each face's index tuple as a nested list into faceVertexIndices and leave faceVertexCounts empty
- B. Extend faceVertexIndices with each face's indices to build one flat list, and append len(indices) per face to faceVertexCounts
- C. Copy the OBJ global index list directly into faceVertexIndices unchanged with no per-face counts
- D. Store the vertex coordinates in faceVertexIndices and the index numbers in faceVertexCounts

Source: Learn OpenUSD: Exercise: Extracting Geometry — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-geometry.html>

Question 11

Domain: Data Modeling

An integration must attach grouped sensor readings (temperature, humidity, pressure) to one prim, but USD has no native struct type. What is the standard convention for such structured data?

- A. Author namespace-prefixed attributes such as acme:sensor:temperature to group the related fields on the prim
- B. Author a relationship per reading that targets a separate prim holding the corresponding value
- C. Define a child prim per reading and author each value as a single attribute on its own prim
- D. Serialize the readings into one string attribute that downstream tools parse back into fields

Source: Learn OpenUSD: Custom Properties — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/custom-properties.html>

Question 12

Domain: Composition

What does the relocates composition arc do, and what constraint applies to the paths it uses?

main.usda

```
#usda 1.0
(
  relocates = {
    </CharANewVersion/Clothing> : </CharACurrent/TestClothing>
  }
)
```

- A. It removes the source prim from the referenced layer and reauthors it at the target path, permanently rewriting the composition source
- B. It copies attribute time samples from a source prim to a target prim, using mappings that may name either prim paths or property paths
- C. It non-destructively renames or reparents a prim brought in across a composition arc, using layer-metadata source-to-target mappings that may be prim paths only, not property paths
- D. It is authored on the prim itself like a reference, and its mappings may remap both prim paths and individual property paths

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

Question 13

Domain: Composition

A prim has a `prepend references` arc that delivers `radius = 5.0` and a `prepend specializes` arc whose source class defines `radius = 2.0` as a fallback. No local opinion for `radius` exists on the prim itself. Under LIVERPS, which value composes as the final result?

- A. `3.5` — USD linearly interpolates competing opinions when neither arc provides a local override to break the tie
- B. `2.0` — the specializes class opinion wins because it is authored closer to the prim in the scene graph than an external reference
- C. `5.0` — the reference opinion wins because references (R) are positioned above specializes (S) in the LIVERPS strength ordering
- D. `2.0` — the specializes fallback wins because class-sourced opinions are treated as the strongest non-local arc in LIVERPS

Source: Learn OpenUSD: Specializes — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/inherits-specializes/what-is-specializes.html>

Question 14

Domain: Visualization

A high-resolution mesh is used only for final rendering while a lightweight stand-in appears in the viewport. Which two purpose token values express this render-versus-viewport distinction?

- A. default and guide
- B. render and proxy
- C. render and guide
- D. inherited and invisible

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

Question 15

Domain: Pipeline Development

A tool authors thousands of attribute edits in a loop and is too slow. Which mechanism is designed to batch these edits for performance?

- A. Call `UsdStage.Flatten` after each edit to commit changes faster
- B. Set a custom `ArResolver` to defer the writes until close
- C. Wrap the edits in an `SdfChangeBlock` to batch change notifications
- D. Switch the edit target to a USDA layer so writes are buffered

Source: NCP-OUSD Exam Study Guide (PDF) — <https://nvdam.widen.net/s/6kxsqcsrrw/ncp-openusd-development-study-guide>

Question 16

Domain: Composition (Select three options.)

How do payloads differ from references in OpenUSD?

payloads_simple.usda

```
#usda 1.0
(
    defaultPrim = "World"
)

def Xform "World"
{
    def "red_cube_01" (
        prepend payload = @./red_cube.usd@
    )
    {
    }
}
```

- A. Payload composition arcs can be applied to the same prim in a list-editable way, just like references.
- B. Payloads are weaker than references in the LIVERPS composition strength order.
- C. Payloads are stronger than references, so a payload opinion overrides a competing referenced opinion.
- D. Payloads can be loaded and unloaded on demand, while references are always composed and present on the stage.
- E. Unloading a payload recomposes the prim but leaves all descendant prims of the payloaded prim loaded.

Source: Learn OpenUSD: Payloads — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-payloads.html>

Question 17

Domain: Data Modeling

A stage starts at time code 1, but an attribute's first authored time sample is at time code 60. What value does the attribute resolve to when queried at time code 1, before the first sample?

clamp.usda

```
#usda 1.0
(
    startTimeCode = 1
    endTimeCode = 120
    timeCodesPerSecond = 30
)

def Cube "AnimCube"
{
    float3 xformOp:scale = (1.5, 1.5, 1.5)
    float3 xformOp:scale.timeSamples = {
        60: (2.5, 2.5, 2.5),
        120: (5, 5, 5),
    }
    uniform token[] xformOpOrder = ["xformOp:scale"]
}
```

- A. (0, 0, 0) — no value resolves before the first sample
- B. An interpolated value between the default and the first sample
- C. (2.5, 2.5, 2.5) — the first (earliest) time sample, held/clamped
- D. (1.5, 1.5, 1.5) — the authored default value

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 18

Domain: Composition

What does the defaultPrim layer metadatum provide, and what happens if it is unset when the layer is referenced?

asset.usda

```
#usda 1.0
(
    defaultPrim = "Asset"
)

def "Asset"
{
    def "Geom"
    {
    }
}
```

- A. It controls which top-level prim is active on the stage; if unset, activation falls back to each prim's authored opinion
- B. It marks the prim that Hydra renders first when the layer is opened; if unset, prims render in their authored namespace order
- C. It names the entry-point prim used when the layer is referenced; if unset, the layer resolves as empty with a warning and usdchecker reports an error
- D. It records the stage's up axis for consuming applications; if unset, each consumer assumes its own default axis

Source: Learn OpenUSD: Default Prim — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/default-prim.html>

Question 19

Domain: Composition

A prim receives an opinion for radius from an inherited class, from a selected variant, and from a reference, with no local opinion present. Which opinion wins?

- A. The opinion in the strongest sublayer, regardless of which arc delivered it
- B. The referenced opinion, because references resolve before inherits
- C. The inherited opinion, because inherits is stronger than variants and references
- D. The variant opinion, because variant selections override inherited class opinions

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

Question 20

Domain: Composition

Two variant sets exist on an asset: 'color' flips an attribute value, and 'lod' swaps which mesh file is referenced. For frequent real-time toggling, which set is preferable and why?

asset.usda

```
#usda 1.0
def Xform "Asset" (
    variantSets = ["color", "lod"]
)
{
    variantSet "color" = {
        "red" { color3f primvars:displayColor = [(1,0,0)] }
        "blue" { color3f primvars:displayColor = [(0,0,1)] }
    }
    variantSet "lod" = {
        "hi" { def "Mesh" ( prepend references = @hi_res.usda@ ) {} }
        "lo" { def "Mesh" ( prepend references = @lo_res.usda@ ) {} }
    }
}
```

- A. 'color', because changing an attribute value only affects value resolution and leaves the composition index intact
- B. Either; all variant switches cost the same since both reconfigure the composition index
- C. 'lod', because changing reference paths is lighter than recomputing an attribute value
- D. 'lod', because swapping referenced files only re-resolves values without rebuilding composition

Source: Learn OpenUSD: Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/variant-sets/what-are-variant-sets.html>

Question 21

Domain: Data Exchange

An obj2usd converter reads an OBJ material that supplies a SHININESS value, but UsdPreviewSurface expects roughness. How is the value mapped, and which workflow flag is set?

extract_material.py

```
roughness = 1 - math.sqrt(mtl["SHININESS"] / 1000.0)
shader.CreateInput("useSpecularWorkflow", Sdf.ValueTypeNames.Int).Set(1)
shader.CreateInput("roughness", Sdf.ValueTypeNames.Float).Set(roughness)
```

- A. Copy shininess into the roughness input unchanged because the two are equivalent
- B. Set roughness equal to shininess / 1000.0 directly and enable the metallic PBR workflow
- C. Author the raw shininess value directly on a shininess input and leave useSpecularWorkflow unset

- D. Convert shininess to roughness with $\text{roughness} = 1 - \sqrt{\text{shininess} / 1000.0}$ and set useSpecularWorkflow to 1

Source: Learn OpenUSD: Exercise: Extracting Materials — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-materials.html>

Question 22

Domain: Data Modeling

An attribute has no authored default and no time samples. What does Get() return for it?

- A. An empty/None value with no fallback
- B. An error, because the attribute is undefined
- C. The nearest authored value from an ancestor prim
- D. The schema-defined fallback value for that attribute

Source: Learn OpenUSD: Attributes — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/properties/attributes.html>

Question 23

Domain: Pipeline Development

During review a TD finds a leaf Mesh (a gprim) deep in an asset tagged kind = "component". Beyond being unusual, why is this a structural problem for model hierarchy?

example.usda

```
def Mesh "Body" (  
    kind = "component"  
) {  
}
```

- A. A gprim tagged as a component signals a deep model hierarchy, adding composition overhead and losing pruned-traversal gains
- B. The component kind is intended for Xform prims, so Hydra skips the Mesh when populating the render index
- C. kind metadata on a gprim is ignored during composition, so the model boundary silently falls back to the asset root
- D. A component kind on a leaf forces every ancestor to compose as a component as well, inverting the model hierarchy

Source: Learn OpenUSD: Model Hierarchy Considerations — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/model-hierarchy-considerations.html>

Question 24

Domain: Customizing USD

A team is deciding which native OpenUSD file format to use for a layer that stores vertex positions, normals, and UV coordinates for a mesh with one million points. Which choice best reflects OpenUSD's production guidance for numerically heavy geometry data?

- A. USDZ, because it bundles the mesh with its referenced textures into one atomic archive suitable for any stage in the pipeline.
- B. USDC, because its binary Crate format uses compression and memory mapping optimized for numerically heavy data, opening faster and using less memory.

- C. USD with a .usd extension, because it automatically selects ASCII or binary encoding based on the size of the data in the layer.
- D. USDA, because its human-readable ASCII encoding makes the vertex data easy to inspect in any text editor and diff in version control.

Source: Learn OpenUSD: OpenUSD File Formats — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/usd-file-formats.html>

Question 25

Domain: Composition

Why are prims authored with the class specifier neither rendered by Hydra nor visited by a default stage traversal, even though they carry attribute opinions?

- A. Because class prims are deactivated by default and must be activated before they can be traversed
- B. Because class prims are converted to over specifiers during composition, leaving nothing concrete to draw
- C. Because class is an abstract specifier meant to serve as a composition arc source, not concrete scene content
- D. Because class prims carry their opinions as metadata rather than as attribute specs renderers can read

Source: Learn OpenUSD: Specifiers — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/specifiers.html>

Question 26

Domain: Customizing USD

Which statements correctly distinguish IsA (typed) schemas from API schemas?

- A. Both IsA and API schemas may each be applied multiple times to a single prim
- B. Abstract IsA schemas like UsdGeomPointBased carry a typeName and can be instantiated directly as prims
- C. API schemas set the prim's typeName, while IsA schemas are applied via the apiSchemas list
- D. A prim has exactly one IsA schema (its typeName), while many API schemas may be applied to the same prim

Source: Learn OpenUSD: Schemas — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/schemas.html>

Question 27

Domain: Content Aggregation *(Select two options.)*

A scene uses scenegraph instancing with many instances sharing one prototype. Which refinement edits, authored on the instanceable prims, cause OpenUSD to create a NEW prototype as a result of the edit?

- A. Adding a new ad hoc internal reference to an instanceable prim
- B. Selecting a different variant on an instanceable prim via a variant set placed there
- C. Authoring an inherited primvar value on the instanceable prim that a shader reads
- D. Authoring xformOp:translate on the instanceable prim to reposition it

Source: Learn OpenUSD: Ad Hoc Arcs Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-ad-hoc-arcs-refinement.html>

Question 28

Domain: Debugging and Troubleshooting

An EditTarget is used to direct authored opinions to a specific contributing site. Which sites can a UsdEditTarget select?

set_target.py

```
stage.SetEditTarget(Usd.EditTarget(sublayer))
# or scope edits to a variant:
with vset.GetVariantEditContext():
    prim.GetAttribute('color').Set((1,0,0))
```

- A. Any reference or payload target in a prim's index, with local sublayer and variant edits routed through the stage's root layer
- B. The currently selected variant of a defined variant set or the session layer, with sublayer edits authored by opening the sublayer as its own stage
- C. Any sublayer in the stage's root layer stack or the session layer, with variant edits authored by opening the variant's source layer as its own stage
- D. Any sublayer in the stage's root layer stack, the currently selected variant of a defined variant set, or any node in a prim's index such as a reference target

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

Question 29

Domain: Data Exchange

A studio architect is documenting when and where data transformation steps may execute within a USD export pipeline. Which statement correctly describes the scheduling flexibility of a transformation step?

- A. A transformation step completes before the extracted stage is accessible to any downstream tool.
- B. A transformation step executes in the same process as the extraction step, before the stage is written to disk.
- C. A transformation step is limited to operations exposed through the DCC application's export dialog.
- D. A transformation step may run as a separate process, a cloud service, or as a post-process after a stage has been exported.

Source: Learn OpenUSD: What Is Data Transformation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/what-is-data-transformation.html>

Question 30

Domain: Data Exchange

An OBJ stores one global vertex list with indices grouped per face, but UsdGeom.Mesh needs a flat faceVertexIndices list recomposed via faceVertexCounts. Iterating each face's index group, which pair of operations assembles both arrays?

- A. Extend faceVertexIndices with the face's indices, and append len(indices) to faceVertexCounts.
- B. Extend faceVertexCounts with the face's indices, and append len(indices) to faceVertexIndices.
- C. Append the face's index group as a sublist to faceVertexIndices, and append the global vertex count to faceVertexCounts.
- D. Append the first index of each face to faceVertexIndices, and set faceVertexCounts to the number of faces.

Source: Learn OpenUSD: Exercise: Extracting Geometry — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-geometry.html>

Question 31

Domain: Pipeline Development

An asset exposes both a color variant set and a primvar on its entry point for downstream parameterization. How do the two approaches differ with respect to scenegraph instancing?

- A. Variant selections and primvar overrides both generate one new prototype per distinct parameterization value.
- B. Variant selections generate new prototypes for each distinct selection, while primvar overrides do not generate new prototypes.
- C. Variant selections and primvar overrides both share a single prototype regardless of how many different values are authored.
- D. Primvar overrides generate new prototypes for each distinct value authored, while variant selections share one prototype across all selections.

Source: Learn OpenUSD: What Is Asset Parameterization? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-parameterization/what-is-asset-parameterization.html>

Question 32

Domain: Pipeline Development

A synthetic-data pipeline distributes a large simulation across ten machines. Each machine writes its portion of results to a separate USD layer. Which pattern is recommended to combine the per-machine outputs into a single composed result?

- A. A single USDC binary layer into which all per-machine output layers are merged at publish time, because layer stacks cannot efficiently compose more than a few layers.
- B. Separate reference arcs, one per machine layer, each grafted under a distinct prim path in a root assembly stage.
- C. `UsdStage::Flatten()` applied to each machine's layer individually, with the flattened results appended sequentially to the primary layer.
- D. A layer stack that sublayers all ten output layers, composing the distributed contributions into a unified asset through the standard sublayer composition.

Source: Learn OpenUSD: Modeling Workstreams With Layer Stacks — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/workstreams/modeling-workstreams.html>

Question 33

Domain: Data Modeling

An `xformOp:scale` has an authored default of `(1.5,1.5,1.5)` plus time samples `{60:(2.5,2.5,2.5),120:(5,5,5)}` on a stage whose `startTimeCode` is 1. Querying `GetXformVectors` at time code 1 returns which scale, and why?

example.usda

```
float3 xformOp:scale = (1.5, 1.5, 1.5)
float3 xformOp:scale.timeSamples = {
    60: (2.5, 2.5, 2.5),
    120: (5, 5, 5),
}
```

- A. `(1.5,1.5,1.5)`, because the authored default value is used before the first time sample
- B. `(1,1,1)`, because the schema fallback applies when no sample exists at the queried time code
- C. `(2.5,2.5,2.5)`, because queries before the first sample clamp to the earliest authored time sample
- D. `(0.04,0.04,0.04)`, because values extrapolate linearly backward from the first two samples

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 34

Domain: Composition

A developer creates a variant set named `material`, adds a `red` variant, selects it, and wants to author a color override that applies only when `red` is the active variant. Which Python call opens the authoring context so that subsequent edits are scoped inside that specific variant?

setup_variants.py

```
vsets = prim.GetVariantSets()
vset = vsets.AddVariantSet("material")
vset.AddVariant("red")
vset.SetVariantSelection("red")
# <--- open variant edit context here, then author opinions
```

- A. `vset.SetVariantSelection("red")` — selecting a variant redirects subsequent authored edits into that variant scope
- B. `vset.GetVariantEditContext()` — the context manager returned by this call directs opinions into the selected variant
- C. `vset.AddVariant("red")` — adding the variant automatically scopes subsequent edits to that variant
- D. `vsets.AddVariantSet("material")` — creating the variant set opens an implicit edit context for its first variant

Source: Learn OpenUSD: Variant Sets Basics — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/variant-sets.html>

Question 35

Domain: Debugging and Troubleshooting

While debugging a prim, you want the list of `PrimSpecs` contributing to the prim's metadata, and separately the list of `PropertySpecs` contributing to one property's value. Which APIs return these two things?

inspect.py

```
prim = stage.GetPrimAtPath('/Char')
prim_specs = prim.GetPrimStack() # PrimSpecs -> prim metadata
attr = prim.GetAttribute('displayColor')
prop_specs = attr.GetPropertyStack(Usd.TimeCode.Default()) # PropertySpecs
```

- A. `UsdPrim::GetPrimStack` returns both the `PrimSpecs` and the `PropertySpecs`, since a property stack is a filtered view of the prim stack
- B. `UsdProperty::GetPropertyStack` returns the `PrimSpecs` for the prim's metadata, and `UsdPrim::GetPrimStack` returns the `PropertySpecs` for the property's value
- C. `UsdPrim::GetPrimStack` returns the `PrimSpecs` for the prim's metadata, and `UsdProperty::GetPropertyStack` returns the `PropertySpecs` for the property's value
- D. `UsdProperty::GetPropertyStack` returns both the `PrimSpecs` and the `PropertySpecs`, recorded on each contributing site it visits

Source: Learn OpenUSD: OpenUSD Glossary — <https://docs.nvidia.com/learn-openusd/latest/glossary.html>

Question 36

Domain: Debugging and Troubleshooting (Select two options.)

When debugging why a composed prim or property has an unexpected value, which statements about the prim stack and property stack are correct?

- A. The prim stack and property stack are the recommended high-performance path for repeatedly resolving an attribute value.
- B. The property stack always contains the correct time-offsets for animated values even when layer offsets are present.
- C. `UsdProperty::GetPropertyStack()` should only be used for debugging, not for value resolution.
- D. `GetPrimStack()` returns only the single strongest prim spec, discarding all weaker contributing specs.
- E. `UsdPrim::GetPrimStack()` returns the ordered list of prim specs across layers that contribute opinions to a prim's metadata.

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

Question 37

Domain: Composition

A character asset defines a variantSet lod whose high variant authors its own prepend references = `@high_res_mesh.usda@`. When does that reference arc participate in composition?

- A. Never, because variants cannot author composition arcs
- B. Only while the high variant is the selected variant
- C. Always, because reference arcs are evaluated independently of variant selection
- D. Only if the reference is also declared outside the variantSet

Source: Learn OpenUSD: Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/variant-sets/what-are-variant-sets.html>

Question 38

Domain: Debugging and Troubleshooting

An edit authoring color = (0,0,1) into `weak.usda` never appears on the composed stage, because `strong.usda` already authors color = (1,0,0). Why is the edit ineffective, and what is the cleanest fix?

root.usda

```
#usda 1.0
(
  subLayers = [
    @strong.usda@,
    @weak.usda@
  ]
)
```

strong.usda

```
#usda 1.0
over "Prop"
{
  color3f color = (1, 0, 0)
}
```

weak.usda

```
#usda 1.0
over "Prop"
{
    color3f color = (0, 0, 1)
}
```

- A. Sublayers listed later in subLayers are stronger, so weak.usda already wins; reloading the stage picks up the edit
- B. The edit went to the stronger sublayer, where root.usda's own opinion masks it; muting root.usda makes it take effect
- C. The edit is masked by a session layer override on color; clearing the session layer lets the weak.usda opinion through
- D. The edit went to the weaker sublayer, so strong.usda wins; authoring to the stronger sublayer or a stronger edit target makes it win

Source: Learn OpenUSD: Sublayers — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/sublayers/what-are-sublayers.html>

Question 39

Domain: Pipeline Development

An asset's root layer is the primary means downstream users interact with it. Which metadata typically designates the single entry point prim that referencing will respect?

- A. The root layer's defaultPrim metadata
- B. The upAxis metadata in the layer header
- C. The kind = component metadata on the deepest gprim
- D. An instanceable = true tag on the World prim

Source: Learn OpenUSD: What Is an Asset Interface? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-structure-principles/asset-interface-pt1.html>

Question 40

Domain: Debugging and Troubleshooting

A pipeline temporarily needs to suppress one layer's contributions to the composed stage without deleting it from the layer stack. Which API does this?

- A. UsdProperty::GetPropertyStack()
- B. UsdStage::Flatten()
- C. SdfChangeBlock()
- D. UsdStage::MuteLayer()

Source: NCP-OUUSD Exam Study Guide (PDF) — <https://nvdam.widen.net/s/6kxsqcsrrw/ncp-openusd-development-study-guide>

Question 41

Domain: Customizing USD

A concrete IsA schema and an abstract IsA schema differ in their schema definitions. What distinguishes a concrete typed schema from an abstract (non-concrete) one?

- A. An abstract schema can be instantiated as prims, while a concrete schema serves only as a base class for derived schemas

- B. A concrete schema is applied through the `apiSchemas` list, while an abstract schema is assigned through the `typeName` metadata
- C. A concrete schema provides a name and a `typeName` and can be instantiated as prims, while an abstract schema has no `typeName` and serves as a base class
- D. A concrete schema is generated as a codeless schema, while an abstract schema ships with compiled C++ and Python classes

Source: Learn OpenUSD: Schemas — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/schemas.html>

Question 42

Domain: Data Modeling

What distinguishes a primvar from an ordinary USD attribute?

- A. An attribute in the primvars: namespace that can be read by shaders, inherits down namespace, and carries an interpolation mode
- B. A property that holds paths to other prims so that geometry can target the material that shades it
- C. A shader input in the inputs: namespace that routes a constant or texture value into a material's surface output
- D. A name-value pair stored in a metadata dictionary that resolves element by element and is authored without time samples

Source: Learn OpenUSD: Primvars — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/primvars.html>

Question 43

Domain: Content Aggregation

A `UsdGeomPointInstancer` authors only prototypes, `protoIndices = [0, 0, 1]`, and positions, with no `ids` attribute. What integer id does the third placed instance receive, and from where?

example.usda

```
rel prototypes = [<Prototypes/A>, <Prototypes/B>]
int[] protoIndices = [0, 0, 1]
point3f[] positions = [...]
```

- A. Id 1, copied from its prototype index value in `protoIndices`
- B. Id 2, inferred from that instance's element index in the `protoIndices` array
- C. Id 3, a one-based counter assigned in positions order
- D. No id, because ids must be authored explicitly for instances to be addressable

Source: Learn OpenUSD: Introduction to Point Instancing — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/authoring-point-instancing/point-instancing-intro.html>

Question 44

Domain: Pipeline Development

A model hierarchy is described as having a shallowness requirement. Which statement correctly identifies the sign that a model hierarchy has become too deep?

- A. An assembly model is tagged `kind = component`, causing all of its descendants to be treated as subcomponent prims.
- B. A leaf gprim, such as a Mesh, is tagged `kind = component`, placing the traversal-pruning boundary at the geometry level.

- C. An assembly model has more than two levels of group-kind ancestors between it and the scene root, adding unnecessary traversal overhead.
- D. A component model references another asset whose defaultPrim carries no kind metadata, preventing kind-based discovery from finding it.

Source: Learn OpenUSD: Model Hierarchy Considerations — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/model-hierarchy-considerations.html>

Question 45

Domain: Composition

An asset is brought onto /Set/PropA via a reference that sets color = (1,0,0). A sublayer of the root layer authors a direct opinion color = (0,1,0) on /Set/PropA. What color composes, and what rule governs the result?

root.usda

```
#usda 1.0
(
  subLayers = [@local_overrides.usda@]
)

def "Set"
{
  def "PropA" (
    prepend references = @propA_asset.usda@</Prop>
  )
  {
  }
}
```

local_overrides.usda

```
#usda 1.0
over "Set"
{
  over "PropA"
  {
    color3f color = (0, 1, 0)
  }
}
```

- A. (0,0,0), because neither opinion is authored in the root layer itself
- B. (1,0,0), because referenced opinions are evaluated before local sublayer opinions
- C. (0,1,0), because a local opinion in the layer stack beats any referenced opinion
- D. (1,0,0), because the reference is prepended and therefore strongest

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

Question 46

Domain: Debugging and Troubleshooting

Before delivering an asset, a validation pass must flag a missing defaultPrim, missing upAxis, and missing metersPerUnit, among other compliance issues. Which tool performs this validation?

- A. usdchecker
- B. UsdStage::MuteLayer

- C. SdfChangeBlock
- D. TfDebug

Source: Learn OpenUSD: Exercise: Asset Validation and Testing — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/asset-validation/exercise-asset-validation-testing.html>

Question 47

Domain: Debugging and Troubleshooting

A pipeline developer wants to understand whether USD caches composition results or resolved attribute values at stage load time. Which statement correctly describes the caching model?

- A. Both composition and value resolution results are cached at load time for maximum query speed.
- B. Neither composition nor value resolution is cached; all data is read directly from layers on each access.
- C. Value resolution is cached per attribute, but composition is recomputed on every property query.
- D. Composition is cached at the prim level, but value resolution is not pre-calculated.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 48

Domain: Data Exchange

An exported asset fails when referenced into a shot: the material binding cannot resolve. Inspecting the layer, the binding target points to a prim outside the asset's defaultPrim hierarchy. What is the root cause and fix?

asset.usda

```
#usda 1.0
(
  defaultPrim = "Asset"
)

def Xform "Asset"
{
  def Mesh "Body" (
    prepend apiSchemas = ["MaterialBindingAPI"]
  )
  {
    rel material:binding = </Looks/Metal>
  }
}

def Scope "Looks"
{
  def Material "Metal" {}
}
```

- A. The binding targets a prim outside the defaultPrim hierarchy; author the material under the Asset prim so the target resolves when referenced
- B. The Mesh lacks an authored subdivisionScheme; set it to none so the binding relationship resolves on polygonal geometry
- C. The defaultPrim is set to the wrong prim; point it at Looks so the material is composed when the asset is referenced
- D. The apiSchemas entry uses the wrong token; author it as UsdShadeMaterialBindingAPI so the binding relationship is recognized

Source: Learn OpenUSD: Exercise: Transforming the Prim Hierarchy — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/transformation-hierarchy.html>

Question 49

Domain: Data Exchange (Select two options.)

Running `usdchecker` on a freshly converted `obj2usd` asset reports three errors related to stage metadata, then a transformation step is added. Which statements about `usdchecker` and the Omniverse Asset Validator are correct?

- A. The Omniverse Asset Validator implements its own rule engine that is developed separately from `usdchecker`'s rule set.
- B. `usdchecker` automatically rewrites the converter's extraction code to author whatever stage metadata is missing.
- C. The Omniverse Asset Validator is based on `usdchecker`, adding automatic fixes for failed validations and the ability to add new rules.
- D. `usdchecker` is part of the USD Toolset and validates a stage or USDZ package against defined rules for interchange and Hydra renderability.
- E. `usdchecker` validates standalone USDA text layers but has no support for inspecting packaged USDZ archives.

Source: Learn OpenUSD: What Is Asset Validation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/asset-validation/what-is-asset-validation.html>

Question 50

Domain: Data Exchange

An OBJ-to-USD converter authors meshes from OBJ data, which has no subdivision-surface concept. The converter sets the subdivision scheme as shown. What is the effect of this authored value, and why is it required?

example.py

```
usd_mesh.CreateSubdivisionSchemeAttr(UsdGeom.Tokens.none)
```

- A. It treats each mesh explicitly as a polygonal mesh rather than a SubD surface, which is required because OBJ has no subdivision concept.
- B. It restricts the mesh to quad topology where every `faceVertexCount` equals four, which is required because OBJ faces are quadrilaterals.
- C. It authors a display hint that Hydra ignores at render time, which is required for `usdchecker`'s mesh validation rules to pass.
- D. It enables Catmull-Clark smoothing of the imported polygons, which is required because OBJ meshes are implicitly SubD surfaces.

Source: Learn OpenUSD: Exercise: Extracting Geometry — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-geometry.html>

Question 51

Domain: Data Modeling

A developer authors a custom attribute with `custom=True`. What effect does the `custom=True` flag have on how OpenUSD treats that attribute?

- A. It registers the attribute with the schema registry so USD treats it as a builtin property

- B. It marks the attribute as animatable so that USD permits time samples to be authored on it
- C. It prevents opinions in stronger layers from overriding the attribute's authored value
- D. It has no effect on how USD treats the attribute; it signals the attribute is not part of a schema

Source: Learn OpenUSD: Custom Properties — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/custom-properties.html>

Question 52

Domain: Composition (Select two options.)

Which statements about `UsdStage::Flatten()` are correct?

- A. Flattening discards the stage's defaultPrim metadata from the output layer
- B. Flattening composes the whole stage into a single layer, resolving all composition arcs
- C. Flattening resolves local opinions while leaving inherits and variants as live arcs
- D. Flattening a stage is a distinct operation from flattening a layer stack, which uses `UsdUtilsFlattenLayerStack`
- E. Flattening preserves every reference and payload arc unresolved in the output layer

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

Question 53

Domain: Composition

OpenUSD recently introduced a new composition arc that added a letter to the strength-ordering acronym, turning LIVRPS into LIVERPS. What is this arc and where does it sit in the strength ordering?

- A. References; the 'E' distinguishes external references, which sit between inherits and variant sets
- B. Relocates; it is the 'E' (rElocates) and sits ahead of local opinions as the strongest arc
- C. Specializes; the 'E' marks an 'extends' arc that becomes the new weakest arc below specializes
- D. Relocates; it is the 'E' (rElocates) and sits between variant sets and references in the ordering

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

Question 54

Domain: Content Aggregation

A developer sees `"/__Prototype_01"` in the usdview tree and authors the override below to recolor it. Why does this fail to affect the actual instances?

example.usda

```
over "__Prototype_01" {
    color3f primvars:asset_color = [(1, 0, 0)]
}
```

- A. The override uses constant interpolation, while primvars must use vertex interpolation to reach the instances
- B. The override needs instanceable set to false on the prototype before its opinions can compose onto instances
- C. The override targets a runtime data model that is not editable, leaving a dangling spec unrelated to the real prototype
- D. The override is composed, but the prototype's opinions are weaker than the instanceable prim's own opinions

Source: Learn OpenUSD: Introduction to Scenegraph Instance Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-instance-refinement.html>

Question 55

Domain: Data Exchange

A proprietary CAD application keeps geometry in its own native format as the single source of truth and must compose directly into a USD stage without a pre-export step. Which OpenUSD integration mechanism uniquely enables this, and which capability makes it the right choice?

- A. A file format plugin, because the native file can be referenced, sublayered, or loaded as a payload into a USD stage without a prior conversion step.
- B. A DCC exporter, because it translates the application's runtime format to OpenUSD and supports bidirectional conversion.
- C. A standalone converter, because it can be run as a batch process that translates the native format to USD at composition time.
- D. A DCC importer, because it translates data from the application's runtime format into OpenUSD structures on demand.

Source: Learn OpenUSD: What Is Data Exchange? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-exchange/what-is-data-exchange.html>

Question 56

Domain: Data Modeling *(Select three options.)*

Two sublayers author the same prim, with the stronger layer listed first. Which outcomes correctly describe how OpenUSD value resolution combines the different data types on that prim?

- A. Metadata of every type combines element by element, regardless of the value's data type.
- B. Dictionary metadata like customData combines element by element, so keys authored only in the weaker layer still appear.
- C. Relationship targets combine opinions from all layers following list-editing rules.
- D. Relationship targets resolve to the strongest layer's target list, discarding the weaker layer's targets.
- E. For most metadata, the strongest opinion wins during composition.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

Question 57

Domain: Pipeline Development

A pipeline tool must compose an entire stage down to a single layer for delivery, removing composition arcs and proprietary file-format dependencies. Which operation is correct, and what distinction matters?

- A. SdfChangeBlock collapses the stage's composition arcs into one layer when the block closes; this differs from flattening a layer stack
- B. UsdStage.Flatten composes the whole stage to one layer and removes arcs; this differs from flattening a single layer stack
- C. Setting the edit target to the root layer bakes composed opinions into that layer; this differs from flattening the full stage
- D. UsdUtilsFlattenLayerStack composes the whole stage into one layer; UsdStage.Flatten merges the sublayers of the root layer stack

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

Question 58

Domain: Content Aggregation (Select two options.)

You reference one asset into two instanceable prims and must recolor an interior Mesh on only one of them, but authoring inside the instance fails because instance subgraphs are read-only. Which approaches work?

- A. Author the color override on the shared prototype prim so it composes into that instance
- B. Set `instanceable = false` on that prim, then author the color on the now-editable interior Mesh
- C. Author the color override on the instance proxy path of the interior Mesh in a stronger layer
- D. Add the interior Mesh's id to `invisibleIds` so the instancer substitutes a recolored prototype
- E. Author a variant set on the instanceable prim whose selection switches the interior color

Source: Learn OpenUSD: Introduction to Scenegraph Instance Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-instance-refinement.html>

Question 59

Domain: Content Aggregation (Select three options.)

A scene uses broadcasted refinement via a specialize arc so that all boxes on one pallet share a common class namespace target. A single reference is added to that class target. Which three statements correctly describe the outcome? (Select three.)

- A. The instance count stays the same because instancing remains active on all boxes throughout the operation.
- B. The prim count increases because the new arc and its referenced content introduce additional prims into the stage.
- C. The total prim count decreases because deinstancing is avoided and fewer per-instance overrides are needed.
- D. No new prototypes are created because broadcasted refinement, like hierarchical primvar refinement, never generates additional prototypes.
- E. A new prototype is created for the affected boxes because their composed subgraph now differs from unmodified boxes.

Source: Learn OpenUSD: Broadcasted Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-broadcasted-refinement.html>

Question 60

Domain: Content Aggregation

For each placed instance, a UsdGeomPointInstancer composes a prototype-root transform, the per-instance scale/orientation/position, and the transform on the PointInstancer prim. Ordered most-local to least-local, which is correct?

- A. Prototype root transform, then scales[i], orientations[i], positions[i], then the PointInstancer prim transform
- B. scales[i], orientations[i], positions[i], then the PointInstancer prim transform, then the prototype root transform
- C. The PointInstancer prim transform, then positions[i], orientations[i], scales[i], then the prototype root transform
- D. positions[i], orientations[i], scales[i], then the prototype root transform, then the PointInstancer prim transform

Source: Learn OpenUSD: Introduction to Point Instancing — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/authoring-point-instancing/point-instancing-intro.html>

Question 61

Domain: Data Exchange (Select two options.)

During the obj2usd transform phase, a chaser reparents all root prims under a new /World prim. Given the snippet, which statements correctly explain why this transformation is needed?

set_default_prim.py

```
def set_default_prim(stage: Usd.Stage):
    root_prims = stage.GetPseudoRoot().GetChildren()
    world_prim = UsdGeom.Xform.Define(stage, "/World").GetPrim()
    stage.SetDefaultPrim(world_prim)
    editor = Usd.NamespaceEditor(stage)
    for prim in root_prims:
        editor.ReparentPrim(prim, world_prim)
    editor.ApplyEdits()
```

- A. The prims that make up the asset shared no common ancestor, so there was no easy way to reference the entire asset.
- B. The converter output had no defaultPrim metadata, so the stage could not be referenced without specifying a target prim.
- C. OBJ already defines a scene-graph root, so the /World prim simply renames the root that extraction produced.
- D. Setting a defaultPrim satisfies the upAxis and metersPerUnit checks that usdchecker reported earlier.
- E. Reparenting the prims under /World lets the geometry compress more efficiently into the USDC crate format.

Source: Learn OpenUSD: Exercise: Transforming the Prim Hierarchy — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/transformation-hierarchy.html>

Question 62

Domain: Visualization (Select three options.)

A lighting artist is working with the UsdLux schema domain. Which statements about UsdLux lights are correct?

lights.usda

```
#usda 1.0

def Scope "Lights"
{
    def DistantLight "Sun"
    {
        color3f inputs:color = (1, 0, 0)
        float inputs:intensity = 120
    }

    def SphereLight "SphereLight"
    {
        color3f inputs:color = (0, 0, 1)
        float inputs:intensity = 50000
    }
}
```

- A. UsdLuxDistantLight represents a directional light, emitting from a distant source along the -Z axis.
- B. UsdLux lights are not Xformable, so they are aimed using shaping parameters rather than transform ops.

- C. UsdLux provides standardized light types such as DistantLight, RectLight, DiskLight, SphereLight, and DomeLight.
- D. UsdLux light types are concrete subclasses of UsdGeomMesh that render as emissive geometry by default.
- E. UsdLux has API schemas that let you add light behavior to existing prims such as meshes and volumes.

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

Question 63

Domain: Debugging and Troubleshooting

After referencing an asset, a material binding stops working and USD warns that a relationship target 'refers to a path outside the scope of the reference ... Ignoring.' What is the most likely root cause and fix?

- A. The binding uses bindMaterialAs weakerThanDescendants, so a descendant's binding wins after composition; strengthening the binding on the referencing side restores it
- B. The bound material was not encapsulated under the referenced source prim; re-author the asset so dependent prims such as the material live under the default prim
- C. The relationship target was authored as a relative path that re-resolved against the wrong prim; converting the target to an absolute stage path repairs the binding
- D. The reference maps the asset's default prim, but the bound material sits under a deactivated ancestor; reactivating that ancestor restores the target

Source: Learn OpenUSD: Exercise: Experimenting With Encapsulation — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/experimenting-with-encapsulation.html>

Question 64

Domain: Visualization

A UsdLuxDistantLight is authored with only xformOp:rotateXYZ = (45, 0, 0) and no other transforms. Before any transform is applied, along which local axis does the DistantLight emit by default?

example.usda

```
def DistantLight "SunLight"
{
    float3 xformOp:rotateXYZ = (45, 0, 0)
    uniform token[] xformOpOrder = ["xformOp:rotateXYZ"]
}
```

- A. Along the local +Y axis, consistent with the Y-up default coordinate system
- B. Along the local +Z axis, representing emission away from the default viewing plane
- C. Along the local -Z axis, representing emission toward the default gaze direction
- D. Along the local +X axis, consistent with the horizontal orientation of a sun light

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

Question 65

Domain: Data Exchange (Select two options.)

A reviewer runs usdchecker on a delivered asset and it reports problems with metadata required for interchange. Which missing fields would usdchecker flag?

- A. A material authored with UsdPreviewSurface instead of a custom shader

- B. A missing defaultPrim on the root layer
- C. A Mesh whose subdivisionScheme is set to none
- D. A missing upAxis or metersPerUnit on the stage
- E. A USDA layer using ASCII instead of Crate encoding

Source: Learn OpenUSD: Exercise: Asset Validation and Testing — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/asset-validation/exercise-asset-validation-testing.html>

Question 66

Domain: Composition

An inherit arc targets an unencapsulated class outside the referenced prim's hierarchy. When the asset is referenced elsewhere and that class is overridden, what is the effect, and what is this pattern called?

- A. Local refinement: the class path is re-pathed per reference, so the override affects only one referenced instance
- B. Global refinement: the unencapsulated class path is not re-pathed on reference, so the override broadcasts to every prim that inherits it
- C. Destructive edit: the override is written back into the asset's source layer, so the original file on disk carries the new opinions
- D. Broken arc: the inherit target fails to resolve once the asset is referenced, because the class lies outside the referenced hierarchy

Source: Learn OpenUSD: Exercise: Experimenting With Encapsulation and Refinement — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/inherits-specializes/experimenting-with-encapsulation-refinement.html>

Question 67

Domain: Composition

A USD layer file contains two top-level prims — `Car` and `Building` — but its `defaultPrim` layer metadata is not set. Another stage references this layer without specifying an explicit target prim path. What is the result?

- A. USD automatically selects the first top-level prim encountered alphabetically, so `Building` is grafted in as the reference target
- B. Both `Car` and `Building` are grafted in as children of the referencing prim, because USD falls back to composing all top-level prims when `defaultPrim` is absent
- C. The reference resolves as an empty layer and a warning is emitted, because no `defaultPrim` is set and no target prim path was supplied
- D. The stage refuses to open, because a missing `defaultPrim` in any referenced layer is a fatal composition error

Source: Learn OpenUSD: Default Prim — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/default-prim.html>

Question 68

Domain: Composition

An aggregate layer needs to include the same skyscraper asset twice and defer loading a heavy city block. Which composition choices fit each need, given how sublayers, references, and payloads behave?

- A. Reference the skyscraper twice and reference the city block, since references can be loaded and unloaded on demand
- B. Payload the skyscraper twice and sublayer the city block, since sublayers defer loading

- C. Reference the skyscraper twice (sublayering would overwrite it), and payload the city block so it can be loaded on demand
- D. Sublayer the skyscraper twice and reference the city block, since references defer loading

Source: Learn OpenUSD: Payloads — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-payloads.html>

Question 69

Domain: Visualization (Select two options.)

A material author wants the same UsdPreviewSurface material to be easily customizable so downstream artists can tweak its look without editing the shading graph. Which practices best support this?

- A. Author every shader input as a fixed constant value inside the shading graph
- B. Expose key shader inputs such as diffuseColor and roughness as overridable parameters on the material
- C. Connect the shader's surface output to the material prim's surface output
- D. Create a separate material for each color and roughness combination in advance
- E. Set the look at render time through renderer-specific overrides instead of USD opinions

Source: Learn OpenUSD: Exercise: Extracting Materials — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-materials.html>

Question 70

Domain: Content Aggregation

In a UsdGeomPointInstancer, what is the correct composition order of transforms applied to each placed instance, from most-local to least-local?

- A. PointInstancer prim transform, then per-instance scale/orient/position, then prototype root transform
- B. Prototype root transform, then per-instance scale/orient/position, then the PointInstancer prim transform
- C. Per-instance scale/orient/position, then prototype root transform, then PointInstancer prim transform
- D. Prototype root transform, then PointInstancer prim transform, then per-instance scale/orient/position

Source: OpenUSD API: UsdGeomPointInstancer — https://openusd.org/dev/api/class_usd_geom_point_instancer.html

Answer Key & Explanations

1. Answer: B [Visualization • obj Hydra architecture — three main parts]

The corpus states: 'A scene delegate, which provides the scene information; The render index, which keeps track of changes and manages the scene; The render delegate, which uses the render index and scene delegate to visualize scene information to create the final image.' The other options swap roles, rename parts, or use non-USD terminology.

Source: Learn OpenUSD: Hydra — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/hydra.html>

2. Answer: C [Customizing USD • obj 5.6]

Deactivating a prim is a non-destructive operation. The prim and its scene description still exist in the composed scenegraph; it is only excluded from the default traversal predicate (`UsdPrimDefaultPredicate`). `GetPrimAtPath()` can always retrieve a valid prim object for any composed prim path, regardless of active status. The default `stage.Traverse()` does not visit inactive prims, so the first option is wrong. `GetPrimAtPath()` does not return `None` for inactive prims—it returns `None` only if the path does not correspond to any prim. Reactivation is not required to access the prim object through the API.

Source: Learn OpenUSD: Active and Inactive Prims — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/active-inactive-prims.html>

3. Answer: A, C [Pipeline Development • obj 7.6]

The guidance is to keep layer stacks simple and manageable, avoiding workstreams that might grow procedurally over time because each layer carries a resolve-and-open cost, and to have each workstream author only its unique contributions sparsely rather than needlessly overwriting another workstream's work. Layer stacks are explicitly not a replacement for asset versioning systems, so capturing versions as sublayers is wrong. Simple assets can live in a single layer, so pre-splitting them is wrong. And computational workstreams can be dynamic, not necessarily consistent between evaluations.

Source: Learn OpenUSD: Modeling Workstreams With Layer Stacks — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/workstreams/modeling-workstreams.html>

4. Answer: D [Composition • obj 1.4]

Omitting the file identifier makes this an internal reference: it targets a prim in the same layer and grafts `/Master's` content onto `/World/InstanceA` with path translation. The arc is authored with the `references` keyword, so it is not an inherit broadcasting opinions, and it is not a payload, which is the arc that defers loading until requested. It also resolves despite `defaultPrim` being `/World`, because an explicit target prim path removes any reliance on the default prim.

Source: Learn OpenUSD: Referencing Basics — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/references.html>

5. Answer: C [Pipeline Development • obj 7.4]

`component` denotes a leaf model and pruning point, so component models cannot contain other component models as descendants; tagging the Cube "Body" (which sits beneath the component "Component") with `kind = component` would make the hierarchy invalid. The other changes are all legal: "Component" already validly carries component under a group ancestor, since ancestors must be group or a subkind of group such as assembly (not assembly specifically, and group is required rather than forbidden), and descendants of a component are expected to be untagged because they sit below the pruning boundary.

Source: Learn OpenUSD: What Are Model Kinds? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/what-are-model-kinds.html>

6. Answer: A [Data Modeling • obj 5.4]

USD linearly interpolates between time samples. At frame 12, $t = (12-1)/(24-1) = 11/23$, giving $x = 11$. The samples are bracketing, so neither clamping nor the fallback applies.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

7. Answer: B [Data Modeling • obj 5.1]

When vertex positions (points) change, the local-space bounding box (extent) can become stale and must be recomputed to keep it synchronized. Because topology is unchanged, faceVertexCounts and faceVertexIndices remain valid and need no update.

Source: Learn OpenUSD: Attributes — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/properties/attributes.html>

8. Answer: D [Visualization • obj 8.3]

A UsdLuxDistantLight emits from a distant source along its local -Z axis; the authored rotation reorients that emission direction. A distant light is directional, not an omnidirectional emitter, and +Z or +Y point opposite or perpendicular to its actual default direction.

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

9. Answer: C [Pipeline Development • obj 7.2]

The rule of thumb is invisibleIds for instances that must animate on and off over time and inactiveIds for the permanent prune, and inactiveIds is the one that supports sparse list-editing of ids. The assignment that gives invisibleIds the permanent prune and inactiveIds the animated subset is exactly reversed. protoIndices and ids serve prototype assignment and identification, not hiding or pruning. And inactiveIds alone cannot cover the animated case, because it is not the time-sampled visibility mechanism.

Source: Learn OpenUSD: Refining Point Instances — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instantiating/refining-point-instances.html>

10. Answer: B [Data Exchange • obj 4.6]

USD prefers a flattened index list plus an ordered list of face vertex counts. The converter extends faceVertexIndices with each face's indices (flattening them into one continuous list) and appends len(indices) per face to faceVertexCounts, so the faces can be recomposed. USD does not accept nested per-face lists, the OBJ grouping must be flattened (not copied verbatim), and the two arrays hold indices and counts, not coordinates.

Source: Learn OpenUSD: Exercise: Extracting Geometry — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-geometry.html>

11. Answer: A [Data Modeling • obj 5.1]

Since USD has no native struct type, namespace-prefixed attributes (acme:sensor:temperature, acme:sensor:humidity, acme:sensor:pressure) are the standard convention for grouped data: the nested prefixes keep related fields together on one prim and identify their origin. Splitting fields across child prims or pointing at value prims through relationships scatters the data instead of grouping it on the prim, and serializing into one string hides the values from typed access.

Source: Learn OpenUSD: Custom Properties — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/custom-properties.html>

12. Answer: C [Composition • obj 1.8]

Relocates non-destructively maps a prim path from a remote layer stack to a new path in the local namespace, authored as source-to-target mappings in layer metadata, and those paths can only be prim paths, not property paths. It does not copy time samples between prims, and property paths are never valid in the mapping. It does not remove or rewrite the composition source, because the operation is non-destructive. And it lives in layer metadata, not on the prim the way a reference is authored.

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

13. Answer: C [Composition • obj 1.8]

In LIVERPS the strength order from strongest to weakest is: Local, Inherits, Variant sets, rElocates, References, Payloads, Specializes. Specializes ('S') is the weakest arc in the entire ordering; it supplies only a fallback value that wins only if no other arc contributes an opinion. Because the reference arc ('R') sits above specializes in the ordering, its `radius = 5.0` opinion wins. Specializes is intentionally designed as the last resort fallback, analogous to a base-class default that is overridden by anything more specific.

Source: Learn OpenUSD: Specializes — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/inherits-specializes/what-is-specializes.html>

14. Answer: B [Visualization • obj 8.1]

The purpose token render marks geometry for final rendering and proxy marks the lightweight viewport stand-in. inherited and invisible are visibility values, not purpose values, and guide is for non-rendered annotation helpers.

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

15. Answer: C [Pipeline Development • obj 7.7]

SdfChangeBlock batches edits, suppressing per-change notification overhead when authoring many changes, which improves performance. Flatten composes the whole stage and is not a per-edit commit, the edit target format does not buffer writes, and ArResolver manages asset path resolution, not write batching.

Source: NCP-OUUSD Exam Study Guide (PDF) — <https://nvdam.widen.net/s/6kxsqcsrrw/ncp-openusd-development-study-guide>

16. Answer: A, B, D [Composition • obj 1.6]

Payloads can be loaded and unloaded on demand whereas references are always composed and present on the stage, payloads are weaker than references in the LIVERPS strength order, and payload arcs can be applied to the same prim in a list-editable way just like references. The reversed strength claim is wrong because payloads sit below references, so a payload opinion does not override a competing referenced opinion. And unloading a payload recomposes the prim and removes the descendants of the payloaded prim from the stage, so they are not left loaded.

Source: Learn OpenUSD: Payloads — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-payloads.html>

17. Answer: C [Data Modeling • obj 5.4]

When a time before the first authored sample is queried, USD clamps to and returns the first (earliest) time sample value, (2.5, 2.5, 2.5) here. Because time samples exist, they win over the authored default during value resolution at a non-Default time code. The default (1.5, 1.5, 1.5) is only returned for Get(Usd.TimeCode.Default()).

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

18. Answer: C [Composition • obj 1.10]

defaultPrim is layer metadata naming the top-level entry-point prim used when the layer is referenced or payloaded; if it is unset, referencing the layer resolves it as empty with a warning, and usdchecker reports an error. The up axis is recorded by the separate upAxis metadatum, not defaultPrim. defaultPrim plays no role in render order, so it does not decide which prim Hydra draws first. And it does not govern prim activation, which is controlled by the active metadata authored on prims.

Source: Learn OpenUSD: Default Prim — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/default-prim.html>

19. Answer: C [Composition • obj 1.7]

LIVERPS orders opinions Local, Inherits, Variants, rLocates, References, Payloads, Specializes. With no local opinion, the inherited opinion is the strongest remaining and wins over both the variant and the reference. References resolve after inherits, not before; variant selections sit below inherits in strength rather than overriding them; and strength is decided by arc class first and then order within a class, never by which sublayer is strongest irrespective of arc type.

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

20. Answer: A [Composition • obj 1.7]

Lightweight variant switches change only resolved values such as an attribute value, so the composition index is unchanged, making the 'color' set preferable for frequent toggling. The 'lod' set swaps reference paths, which is a heavyweight switch because USD must rebuild parts of its composition index, so any option preferring 'lod' as the lighter or composition-preserving choice inverts which operation is heavyweight. The cost is not the same for both, since value-only changes are far cheaper than structural ones.

Source: Learn OpenUSD: Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/variant-sets/what-are-variant-sets.html>

21. Answer: D [Data Exchange • obj 4.6]

The converter derives roughness from the OBJ shininess using $\text{roughness} = 1 - \sqrt{\text{shininess} / 1000.0}$ and sets `useSpecularWorkflow` to 1, because the specular workflow maps most directly to the data coming from OBJ. `UsdPreviewSurface` has no raw shininess input, the values are not equivalent, and a metallic/PBR workflow is not the directly-mapped choice here.

Source: Learn OpenUSD: Exercise: Extracting Materials — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-materials.html>

22. Answer: D [Data Modeling • obj 5.3]

When no value is authored, `Get()` returns the schema fallback value defined for that attribute. USD does not error; attributes are well-defined by their schema. There is no namespace inheritance for ordinary attributes.

Source: Learn OpenUSD: Attributes — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/properties/attributes.html>

23. Answer: A [Pipeline Development • obj 7.3]

A `gprim` tagged as a component is the explicit sign of a deep model hierarchy, which introduces a small but measurable composition overhead and forfeits the performance gains of pruned traversals. `kind` is not restricted to `Xform` prims and Hydra does not skip `kind`-tagged `gprims`. `kind` is read during composition rather than ignored, and there is no fallback of the model boundary to the asset root. And a component tag does not propagate to or convert ancestors, whose kinds are authored independently.

Source: Learn OpenUSD: Model Hierarchy Considerations — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/model-hierarchy-considerations.html>

24. Answer: B [Customizing USD • obj 5.7]

USDC's binary Crate format is explicitly designed to be efficient for numerically heavy data such as geometry; it uses compression and memory mapping to open faster and consume less memory than ASCII USDA. USDA is recommended for small, human-readable interface or aggregation layers that reference or sublayer other files, not for dense vertex arrays. USDZ is a read-only zipped archive intended for final delivery of a complete asset, not for layers being actively iterated on in a pipeline. The `.usd` extension is flexible but does not automatically choose the most efficient encoding—selecting `.usdc` makes the binary choice explicit.

Source: Learn OpenUSD: OpenUSD File Formats — <https://docs.nvidia.com/learn-openusd/latest/stage-setting/usd-file-formats.html>

25. Answer: C [Composition • obj 1.6]

`class` marks an abstract prim: a blueprint intended as the source of inherits or specializes arcs rather than concrete scene content, so Hydra and default traversals skip it while its opinions remain available to broadcast. Class specs are not converted to over during composition; the specifiers remain distinct. Class prims author ordinary attribute specs, not metadata-encoded opinions, which is how they carry values to broadcast. And they are not deactivated by default, since deactivation is a separate mechanism using the active metadata.

Source: Learn OpenUSD: Specifiers — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/specifiers.html>

26. Answer: D [Customizing USD • obj 3.2]

A prim is exactly one `IsA` type, set by its `typeName`, while many API schemas can be layered onto the same prim through the `apiSchemas` list. Abstract typed schemas like `UsdGeomPointBased` provide no `typeName` and cannot be instantiated directly, so the claim that they carry a `typeName` is wrong. A prim cannot subscribe to multiple `IsA` schemas, so the multiple-application claim is wrong. And the assignment mechanisms are inverted in the remaining claim: `IsA` schemas set `typeName`, API schemas go in `apiSchemas`.

Source: Learn OpenUSD: Schemas — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/schemas.html>

27. Answer: A, B [Content Aggregation • obj 2.4]

Variant-set and ad hoc arc refinements change an instance's composition, so OpenUSD generates a new prototype for the affected instances. `Primvar` and `xformOp` refinements only add opinions on the instanceable prim and create no new prototype.

Source: Learn OpenUSD: Ad Hoc Arcs Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-ad-hoc-arcs-refinement.html>

28. Answer: D [Debugging and Troubleshooting • obj 6.2]

A UsdEditTarget can select any sublayer in the stage's root layer stack, the currently selected variant of any defined variant set, or any node in a prim's index, which allows targeting inherited classes, reference targets, and similar sites. Variant edits do not require opening the variant's source layer as its own stage, because variants are directly selectable through the edit target. Sublayer edits likewise do not require opening the sublayer as a separate stage, because sublayers are directly selectable. And local sublayer and variant edits are not routed through the root layer; the edit target addresses each contributing site directly.

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

29. Answer: D [Data Exchange • obj 4.6]

The source explicitly states that a transformation step does not need to happen in the same process as extraction or other transformation steps—it could be a separate process, a cloud service, or a post-process applied after a stage has been exported or converted. The option requiring in-process execution before disk write is an unnecessary restriction not described by the source. The option limiting steps to the DCC export dialog describes one surface where transformation options may appear, but the source describes transformation steps as extendable far beyond that boundary. The option requiring completion before the extracted stage is accessible inverts the pipeline: the extracted stage is the input to transformation, not something gated behind it.

Source: Learn OpenUSD: What Is Data Transformation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/what-is-data-transformation.html>

30. Answer: A [Data Exchange • obj 4.2]

Each face's indices are flattened into one continuous faceVertexIndices list via extend(indices), and the count of indices in that face is recorded by appending len(indices) to faceVertexCounts, which lets USD recompose faces from the flat list.

Source: Learn OpenUSD: Exercise: Extracting Geometry — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-geometry.html>

31. Answer: B [Pipeline Development • obj asset parameterization — variant sets vs primvars and scenegraph instancing]

The page states directly: 'Variations in variant selections will generate new prototypes for downstream contexts, while primvars will not.' This makes parameterization through primvars the lighter choice for single property parameters, offering upfront memory savings at the cost of additional lookups in materials. The option inverting the two behaviors misattributes prototype generation to primvars. The option claiming both generate prototypes per value is contradicted for primvars. The option claiming both share a single prototype is contradicted for variant selections.

Source: Learn OpenUSD: What Is Asset Parameterization? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-parameterization/what-is-asset-parameterization.html>

32. Answer: D [Pipeline Development • obj 7.1]

The workstreams guidance describes computational workstreams where work is partitioned across multiple processes or machines, and a layer stack is then used to stitch the results back together through sublayer composition. Each machine contributes its portion as a sublayer, and composition assembles the distributed outputs into a single result. Merging into a single USDC, using references per prim, or calling Flatten per layer would lose workstream separation or introduce incorrect composition semantics.

Source: Learn OpenUSD: Modeling Workstreams With Layer Stacks — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/workstreams/modeling-workstreams.html>

33. Answer: C [Data Modeling • obj 5.3]

Once time samples are authored they take precedence over the default for timed queries, and a query before the first sample clamps to the earliest authored sample, (2.5,2.5,2.5). The schema fallback applies when nothing at all is authored, the authored default is returned for default-time queries rather than timed ones, and USD holds rather than extrapolates outside the sampled range.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

34. Answer: B [Composition • obj 1.6]

``GetVariantEditContext()`` returns a context object intended for use in a Python ``with`` block. All specs authored inside that context are stored within the selected variant, so they only apply when that variant is active.

``SetVariantSelection`` records which variant is chosen but does not redirect the edit target; ``AddVariant`` merely registers the variant name; and ``AddVariantSet`` creates the set without scoping any edits to a variant.

Source: Learn OpenUSD: Variant Sets Basics — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/variant-sets.html>

35. Answer: C [Debugging and Troubleshooting • obj 6.3]

`UsdPrim::GetPrimStack` returns the prim specs contributing opinions to a composed prim's metadata, while `UsdProperty::GetPropertyStack` returns the property specs contributing a property's value; they are distinct APIs at different namespace levels. The assignment that puts the `PrimSpecs` on `GetPropertyStack` and the `PropertySpecs` on `GetPrimStack` is exactly reversed. Neither API returns both lists: the property specs returned by `GetPropertyStack` do not bundle the prim-level specs, and a property stack is not a filtered view of the prim stack.

Source: Learn OpenUSD: OpenUSD Glossary — <https://docs.nvidia.com/learn-openusd/latest/glossary.html>

36. Answer: C, E [Debugging and Troubleshooting • obj 6.2]

`GetPrimStack()` returns the ordered list of prim specs across layers that contribute opinions to a prim's metadata, and `GetPropertyStack()` should only be used for debugging, not for value resolution. The property stack does not contain the correct time-offsets for animated values when layer offsets are present, so that claim is wrong. For optimized repeated value resolution you should retain a `UsdAttributeQuery` rather than use the stacks. And `GetPrimStack()` returns all contributing prim specs in strength order, not just the single strongest one.

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

37. Answer: B [Composition • obj 1.9]

A variant can author its own composition arcs, including references, but those arcs exist only while that variant is selected, so the `high_res_mesh` reference composes only when the high variant is the active selection. The claim that variants can never author composition arcs is false, as is the idea that reference arcs evaluate independently of variant selection. There is no requirement to also declare the reference outside the `variantSet`.

Source: Learn OpenUSD: Variant Sets — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/variant-sets/what-are-variant-sets.html>

38. Answer: D [Debugging and Troubleshooting • obj 6.2]

`weak.usda` is listed after `strong.usda` in `subLayers`, making it the weaker sublayer, so its color opinion loses to `strong.usda`; the cleanest fix is to author into the stronger sublayer or use a stronger edit target. There is no session layer override in this scenario, so clearing the session layer changes nothing. Sublayers listed earlier in `subLayers` are stronger, not weaker, so `weak.usda` does not already win, and reloading the stage cannot change strength ordering. And the edit demonstrably went to `weak.usda`, not the stronger sublayer, and `root.usda` authors no color opinion that could mask anything.

Source: Learn OpenUSD: Sublayers — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/sublayers/what-are-sublayers.html>

39. Answer: A [Pipeline Development • obj 7.4]

Most assets are organized around defined entry-point prims that act as the interface. A single asset entry point can typically be specified using the root layer's `defaultPrim` metadata, which OpenUSD's composition engine respects when referencing. Library assets with many entry points may instead organize them under scope prims.

Source: Learn OpenUSD: What Is an Asset Interface? — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/asset-structure-principles/asset-interface-pt1.html>

40. Answer: D [Debugging and Troubleshooting • obj 6.1]

`UsdStage::MuteLayer()` temporarily suppresses a layer's contributions to composition without removing it from the layer stack, which is ideal for isolating whether a layer is the source of an opinion. `GetPropertyStack` inspects the ordered list of opinions contributing to a property but suppresses nothing, `SdfChangeBlock` batches authoring change notifications, and `Flatten` composes the entire stage into a single layer rather than suppressing

one layer's contribution.

Source: NCP-OUUSD Exam Study Guide (PDF) — <https://nvdam.widen.net/s/6kxsqcsrrw/ncp-openusd-development-study-guide>

41. Answer: C [Customizing USD • obj 3.2]

Concrete typed schemas provide both a name and a typeName and can be instantiated as prims (for example UsdGeomMesh), while abstract typed schemas provide a name but no typeName and serve as base classes for related concrete schemas (for example UsdGeomPointBased). The instantiation claim with abstract schemas instantiable and concrete ones serving as base classes is the exact inversion. Codeless versus compiled code generation is orthogonal to the concrete/abstract distinction. And neither kind of typed schema is applied through the apiSchemas list; that mechanism belongs to API schemas.

Source: Learn OpenUSD: Schemas — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/schemas.html>

42. Answer: A [Data Modeling • obj 5.2]

A primvar is an attribute in the primvars: namespace that shaders can read, that inherits down namespace to descendant prims, and that carries an interpolation mode. The name-value pair in a metadata dictionary describes customData-style metadata, not a primvar; the property holding paths to other prims describes a relationship; and the input in the inputs: namespace describes a shader input on a UsdShade prim.

Source: Learn OpenUSD: Primvars — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/primvars.html>

43. Answer: B [Content Aggregation • obj 2.3]

ids is optional; when absent, each instance id is inferred from its element index in the protoIndices array, so the third element (index 2) is instance id 2, regardless of its prototype index value.

Source: Learn OpenUSD: Introduction to Point Instancing — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/authoring-point-instancing/point-instancing-intro.html>

44. Answer: B [Pipeline Development • obj 7.3]

The model hierarchy is designed to prune traversal at the component model boundary, which should sit at a relatively shallow point in the scenegraph. The guidance explicitly names a leaf gprim tagged as a component as the sign that a model hierarchy is 'deep.' When the pruning boundary reaches down to raw geometry, the performance benefits of kind-based traversal pruning are lost, and composition overhead increases.

Source: Learn OpenUSD: Model Hierarchy Considerations — <https://docs.nvidia.com/learn-openusd/latest/asset-structure/model-hierarchy/model-hierarchy-considerations.html>

45. Answer: C [Composition • obj 1.3]

The sublayer opinion is a LOCAL opinion in the root's layer stack, and per LIVERPS the Local class beats any non-local arc, including references, so (0,1,0) wins. Sublayer opinions are still local even though they are not in the root layer file, so the value does not fall back to (0,0,0). The prepend keyword orders the reference relative to other references only and never promotes it above local opinions, and referenced opinions are evaluated after, not before, the local layer stack.

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

46. Answer: A [Debugging and Troubleshooting • obj 6.4]

usdchecker validates an asset and flags compliance problems such as a missing defaultPrim, upAxis, and metersPerUnit. TfDebug emits subsystem debug logging via symbolic flags, SdfChangeBlock batches authoring change notifications, and UsdStage::MuteLayer suppresses a layer's contributions to composition; none of them perform asset compliance validation.

Source: Learn OpenUSD: Exercise: Asset Validation and Testing — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/asset-validation/exercise-asset-validation-testing.html>

47. Answer: D [Debugging and Troubleshooting • obj composition caching vs value resolution — key distinction]

The source states: 'Composition is cached, value resolution is not. When you open a stage or add new scene data, USD creates an index of the composition logic and result at the prim-level for quick access. However, USD doesn't pre-calculate the final values of properties.' The both-cached distractor conflates the two systems. The

neither-cached distractor contradicts the explicit statement that composition is indexed. The value-cached distractor inverts which system is cached.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

48. Answer: A [Data Exchange • obj 4.7]

Because referencing pulls in only the defaultPrim hierarchy, a binding that targets /Looks outside that hierarchy cannot resolve; moving the material under the Asset prim restores encapsulation. The applied schema token MaterialBindingAPI is correct as authored, switching defaultPrim to Looks would make the geometry unreferenceable instead, and subdivisionScheme has no effect on relationship resolution.

Source: Learn OpenUSD: Exercise: Transforming the Prim Hierarchy — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/transformation-hierarchy.html>

49. Answer: C, D [Data Exchange • obj 4.7]

Correct: usdchecker is part of the USD Toolset and validates a USD stage or USDZ package against defined rules to assure interchangeability and Hydra renderability, and the Omniverse Asset Validator is based on usdchecker while adding automatic fixes and the ability to add new rules. Wrong: usdchecker does inspect USDZ packages, the Asset Validator builds on usdchecker's rules rather than maintaining a separate engine, and usdchecker reports problems but never rewrites converter code.

Source: Learn OpenUSD: What Is Asset Validation? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/asset-validation/what-is-asset-validation.html>

50. Answer: A [Data Exchange • obj 4.3]

Authoring subdivisionScheme = none makes USD treat the mesh explicitly as polygonal rather than a subdivision surface, preserving intent since OBJ does not support SubD meshes. The attribute is honored by renderers rather than being a cosmetic hint, it imposes no quad-only topology constraint, and none disables smoothing while catmullClark would enable it.

Source: Learn OpenUSD: Exercise: Extracting Geometry — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-geometry.html>

51. Answer: D [Data Modeling • obj 5.5]

Setting custom=True has no impact on how OpenUSD treats the attribute; it is informational metadata telling end-users and developers that the attribute is not part of an existing schema. It does not register anything with the schema registry, it does not shield the value from stronger-layer overrides, and animatability is independent of the flag - attributes can hold time samples regardless.

Source: Learn OpenUSD: Custom Properties — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/custom-properties.html>

52. Answer: B, D [Composition • obj 1.1]

UsdStage::Flatten() composes the entire stage into one layer with every composition arc resolved into the final result, and it is a different operation from flattening a single layer stack, which is done with UsdUtilsFlattenLayerStack. Because arcs are resolved away, references and payloads are not preserved unresolved in the output. Inherits and variants are likewise resolved rather than left as live arcs, and the defaultPrim metadata is not discarded as part of flattening.

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

53. Answer: D [Composition • obj 1.7]

The new arc is relocates, represented by the 'E' (rElocates) in LIVERPS: Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes, so it sits between variant sets and references. It is not the strongest arc, since local opinions remain strongest. The 'E' is not an 'extends' arc below specializes, which remains the weakest arc. And it has nothing to do with external references, which do not move between inherits and variant sets or change the acronym.

Source: Learn OpenUSD: What Is LIVERPS? — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/strength-ordering/what-is-liverps.html>

54. Answer: C [Content Aggregation • obj 2.4]

The `/__Prototype_NN` paths shown in usdview are a runtime data model exposed for inspection only; prototypes are not editable, so the over becomes a dangling spec unrelated to the actual prototype. The failure has nothing to do with primvar interpolation, it is not a composition-strength contest between prototype and instance opinions, and instanceable cannot be toggled on a runtime prototype at all.

Source: Learn OpenUSD: Introduction to Scenegraph Instance Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-instance-refinement.html>

55. Answer: A [Data Exchange • obj File format plugins as USD integration mechanism]

The corpus states that file format plugins allow 'An OpenUSD stage can include a source file format directly as a reference, payload or sublayer. The source file format is translated on the fly, while it is read as a USD document,' and crucially that 'The source file format can remain the source of truth.' Standalone converters produce a separate output file and require a pre-export step. DCC importers translate from OpenUSD into the application's runtime format — the opposite direction. DCC exporters translate from the application's runtime format to OpenUSD, also requiring a pre-export step and not retaining the original format as the source of truth.

Source: Learn OpenUSD: What Is Data Exchange? — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-exchange/what-is-data-exchange.html>

56. Answer: B, C, E [Data Modeling • obj 5.5]

For most metadata the strongest opinion wins; relationship targets are combined from all layers following list-editing rules rather than a single winner being picked; and dictionary-valued metadata like `customData` merges per key, so keys authored only in the weaker layer persist. The statement that relationship targets discard the weaker layer's list contradicts list-edit combining, and element-by-element merging applies to dictionary types, not to metadata of every type.

Source: Learn OpenUSD: Value Resolution — <https://docs.nvidia.com/learn-openusd/latest/beyond-basics/value-resolution.html>

57. Answer: B [Pipeline Development • obj 7.5]

`UsdStage.Flatten` composes the entire stage into a single layer, removing composition arcs and any proprietary file-format dependencies, and is distinct from flattening one layer stack (`UsdUtils.FlattenLayerStack`), which merges just the sublayers of a layer stack. The option crediting `UsdUtils.FlattenLayerStack` with whole-stage composition has the two operations swapped. Setting the edit target only chooses where future opinions are authored and bakes nothing. `SdfChangeBlock` batches change notifications during authoring and never merges arcs or layers.

Source: OpenUSD Glossary — <https://openusd.org/release/glossary.html>

58. Answer: B, E [Content Aggregation • obj 2.1]

Deinstancing the single prim makes its subgraph editable so the interior color can be authored, and a variant selection on the instanceable prim is a permitted refinement that yields a differently colored prototype for that instance. Instance proxies are read-only regardless of layer strength, `invisibleIds` is a `PointInstancer` property and does not swap prototypes or materials, and authoring on the prototype would broadcast the change to every instance rather than one.

Source: Learn OpenUSD: Introduction to Scenegraph Instance Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-instance-refinement.html>

59. Answer: A, B, E [Content Aggregation • obj broadcasted refinement — inherit/specialize arc broadcasting and prototype creation]

The source data shows broadcasted refinement raised the prototype count from 3 to 4 — a new prototype was created for the affected boxes because their composed subgraph now differs from unmodified boxes. The instance count remained at 1450, confirming instancing stayed active on all boxes. The prim count rose from 1711 to 1747, confirming that the new arc and its referenced content introduce additional prims. The option denying prototype creation confuses broadcasted refinement with hierarchical primvar refinement, which does not generate new prototypes. The option claiming the total prim count decreases is contradicted by the recorded increase of 36 prims.

Source: Learn OpenUSD: Broadcasted Refinement — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/refining-scenegraph-instances/scenegraph-broadcasted-refinement.html>

60. Answer: A [Content Aggregation • obj 2.3]

Most local is the prototype root transform; then per-instance scales[i], orientations[i], positions[i]; the PointInstancer prim's own transform is applied least locally, letting you move the whole instancer at once.

Source: Learn OpenUSD: Introduction to Point Instancing — <https://docs.nvidia.com/learn-openusd/latest/asset-modularity-instancing/authoring-point-instancing/point-instancing-intro.html>

61. Answer: A, B [Data Exchange • obj 4.4]

Correct: the flat extracted output had no defaultPrim, so the stage could not be referenced without naming a target prim, and even with one set the asset's prims shared no common ancestor, so the whole asset could not be referenced together; /World plus reparenting fixes both. Wrong: OBJ has no scene-graph root so /World is a genuinely new ancestor rather than a rename, reparenting has no effect on crate compression, and defaultPrim is unrelated to the upAxis and metersPerUnit metadata fixed during extraction.

Source: Learn OpenUSD: Exercise: Transforming the Prim Hierarchy — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-transformation/transformation-hierarchy.html>

62. Answer: A, C, E [Visualization • obj 8.3]

UsdLux includes API schemas that add light behavior to existing prims such as meshes and volumes, provides standardized light types including DistantLight, RectLight, DiskLight, SphereLight, and DomeLight, and its DistantLight emits from a distant source along the -Z axis as a directional light. Lights are Xformable - the lesson rotates and translates them via XformCommonAPI rather than aiming them through shaping parameters - and UsdLux light types belong to the UsdLux schema domain rather than subclassing UsdGeomMesh.

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

63. Answer: B [Debugging and Troubleshooting • obj 6.2]

The warning means the binding relationship targets a path outside the scope of the referenced prim: the material was not encapsulated under the referenced source prim, so the fix is to re-author the asset so all dependent prims, including the material, live under the default prim being referenced. The bindMaterialAs strength setting governs conflicts with descendant bindings and would not produce an out-of-scope warning. A deactivated ancestor would prune the material from composition, but the warning explicitly cites a path outside the reference's scope, not an inactive prim. And switching to an absolute stage path does not help, because any target outside the referenced hierarchy is ignored regardless of how the path is authored.

Source: Learn OpenUSD: Exercise: Experimenting With Encapsulation — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/encapsulation/experimenting-with-encapsulation.html>

64. Answer: C [Visualization • obj UsdLuxDistantLight default emission axis]

The corpus states: 'A light is emitted from a distance source along the -Z axis.' This is the defined default emission direction for UsdLuxDistantLight before any transforms are applied. The rotation shown in the snippet rotates the light but does not alter which axis is the default emission axis.

Source: Learn OpenUSD: Lights — <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/lights.html>

65. Answer: B, D [Data Exchange • obj 4.8]

usdchecker flags interchange issues such as a missing defaultPrim and missing upAxis or metersPerUnit. ASCII vs Crate encoding is a valid choice and not flagged, UsdPreviewSurface is the standard preview shader and is fine, and subdivisionScheme none is correct for polygonal meshes.

Source: Learn OpenUSD: Exercise: Asset Validation and Testing — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/asset-validation/exercise-asset-validation-testing.html>

66. Answer: B [Composition • obj 1.6]

This is global refinement: an inherit target outside encapsulation never gets re-pathed on reference, so overriding the class broadcasts to every prim that inherits it within the referencing context. Local refinement is the opposite case, where an encapsulated target is re-pathed per reference and the override stays with one instance. The edit is non-destructive, landing in the stronger layer rather than the asset's source file. And the arc does not break, since unencapsulated inherit targets keep resolving after the asset is referenced, which is exactly what the

pattern exploits.

Source: Learn OpenUSD: Exercise: Experimenting With Encapsulation and Refinement — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/inherits-specializes/experimenting-with-encapsulation-refinement.html>

67. Answer: C [Composition • obj 1.11]

The ``defaultPrim`` layer metadata designates the single top-level prim that will be used as the entry point when the layer is referenced or loaded as a payload without an explicit target prim path. If ``defaultPrim`` is not set and no target prim path is provided, USD cannot determine which prim to graft, so the reference resolves as an empty layer and a warning is written to the log. Tools such as ``usdchecker`` flag a missing ``defaultPrim`` as an error for this reason.

Source: Learn OpenUSD: Default Prim — <https://docs.nvidia.com/learn-openusd/latest/composition-basics/default-prim.html>

68. Answer: C [Composition • obj 1.4]

Adding the same asset twice via sublayers would overwrite it rather than create two copies, so references are required for the duplicated skyscraper, and payloads are like references but can be loaded and unloaded on demand, which is exactly the deferred-loading behavior the city block needs. Options that defer loading via sublayers or references misassign on-demand loading, which belongs to payloads, and references cannot be unloaded on demand the way payloads can.

Source: Learn OpenUSD: Payloads — <https://docs.nvidia.com/learn-openusd/latest/creating-composition-arcs/references-payloads/what-are-payloads.html>

69. Answer: B, C [Visualization • obj 8.2]

Lofting the shader's surface output up to the material prim's surface output produces a well-formed, consumable material, and exposing inputs such as `diffuseColor` and `roughness` as material-level parameters lets downstream artists override the look without touching the shading graph. Relying on renderer-specific overrides leaves nothing customizable in the USD scene description; fixing every input as a constant inside the graph forces artists to edit the graph to change the look; and pre-building a material per combination multiplies assets instead of enabling customization.

Source: Learn OpenUSD: Exercise: Extracting Materials — <https://docs.nvidia.com/learn-openusd/latest/data-exchange/data-extraction/exercise-extracting-materials.html>

70. Answer: B [Content Aggregation • obj 2.3]

The transform stack composes most-local to least-local as: the prototype's own root transform, then the per-instance scale/orientation/position from the instancer arrays, then finally the `PointInstancer` prim's own transform. The other orderings misplace the instancer prim transform or the prototype root and would yield incorrect world placement.

Source: OpenUSD API: `UsdGeomPointInstancer` — https://openusd.org/dev/api/class_usd_geom_point_instancer.html